Algorithms
THIRD EDITION
IN C

Part 5

GRAPH ALGORITHMS

ROBERT SEDGEWICK

Algorithms
THIRD EDITION
in C

PART 5
GRAPH ALGORITHMS

Robert Sedgewick
Princeton University

Addison-Wesley

Boston • San Francisco • New York • Toronto • Montreal
London • Munich • Paris • Madrid
Capetown • Sydney • Tokyo • Singapore • Mexico City

Many of the designations used by manufacturers and sellers to distinguish their products are claimed as trademarks. Where those designations appear in this book and we were aware of a trademark claim, the designations have been printed in initial capital letters or all capitals.

The author and publisher have taken care in the preparation of this book, but make no expressed or implied warranty of any kind and assume no responsibility for errors or omissions. No liability is assumed for incidental or consequential damages in connection with or arising out of the use of the information or programs contained herein.

Copyright © 2002 by Addison-Wesley

All rights reserved. No part of this publication may be reproduced, stored in a retrieval system, or transmitted, in any form or by any means, electronic, mechanical, photocopying, recording, or otherwise, without the prior written permission of the publisher. Printed in the United States of America. Published simultaneously in Canada.

The publisher offers discounts on this book when ordered in quantity for special sales. For more information, please contact: Pearson Education Corporate Sales Division, One Lake Street, Upper Saddle River, NJ 07458, (800) 382-3419, corpsales@pearsontechgroup.com.

Visit us on the Web at www.awl.com/cseng/ .

Library of Congress Cataloging-in-Publication Data

Sedgewick, Robert, 1946 –
Algorithms in C / Robert Sedgewick. — 3d ed.
500 p. 24 cm.
Includes bibliographical references and index.
Contents: v. 2, pt. 5. Graph algorithms
1. C (Computer program language) 2. Computer algorithms.
I. Title.
QA76.73.C15S43 2002
005.13'3—dc21 97-23418
CIP

ISBN 0201316633

Text printed on recycled and acid-free paper.
6 7 8 9 1011 DOC 09 08 07
6th Printing July 2007

Preface

GRAPHS AND GRAPH algorithms are pervasive in modern computing applications. This book describes the most important known methods for solving the graph-processing problems that arise in practice. Its primary aim is to make these methods and the basic principles behind them accessible to the growing number of people in need of knowing them. The material is developed from first principles, starting with basic information and working through classical methods up through modern techniques that are still under development. Carefully chosen examples, detailed figures, and complete implementations supplement thorough descriptions of algorithms and applications.

Algorithms

This book is the second of three volumes that are intended to survey the most important computer algorithms in use today. The first volume (Parts 1–4) covers fundamental concepts (Part 1), data structures (Part 2), sorting algorithms (Part 3), and searching algorithms (Part 4); this volume (Part 5) covers graphs and graph algorithms; and the (yet to be published) third volume (Parts 6–8) covers strings (Part 6), computational geometry (Part 7), and advanced algorithms and applications (Part 8).

The books are useful as texts early in the computer science curriculum, after students have acquired basic programming skills and familiarity with computer systems, but before they have taken specialized courses in advanced areas of computer science or computer applications. The books also are useful for self-study or as a reference for people engaged in the development of computer systems or applications programs because they contain implementations of useful algorithms and detailed information on these algorithms' performance characteristics. The broad perspective taken makes the series an appropriate introduction to the field.

Together the three volumes comprise the *Third Edition* of a book that has been widely used by students and programmers around the world for many years. I have completely rewritten the text for this edition, and I have added thousands of new exercises, hundreds of new figures, dozens of new programs, and detailed commentary on all the figures and programs. This new material provides both coverage of new topics and fuller explanations of many of the classic algorithms. A new emphasis on abstract data types throughout the books makes the programs more broadly useful and relevant in modern object-oriented programming environments. People who have read previous editions will find a wealth of new information throughout; all readers will find a wealth of pedagogical material that provides effective access to essential concepts.

These books are not just for programmers and computer-science students. Nearly everyone who uses a computer wants it to run faster or to solve larger problems. The algorithms that we consider represent a body of knowledge developed during the last 50 years that has become indispensable in the efficient use of the computer for a broad variety of applications. From N-body simulation problems in physics to genetic-sequencing problems in molecular biology, the basic methods described here have become essential in scientific research; and from database systems to Internet search engines, they have become essential parts of modern software systems. As the scope of computer applications becomes more widespread, so grows the impact of basic algorithms, particularly the fundamental graph algorithms covered in this volume. The goal of this book is to serve as a resource so that students and professionals can know and make intelligent use of graph algorithms as the need arises in whatever computer application they might undertake.

Scope

This book, *Algorithms in C, Third Edition, Part 5: Graph Algorithms*, contains six chapters that cover graph properties and types, graph search, directed graphs, minimal spanning trees, shortest paths, and networks. The descriptions here are intended to give readers an understanding of the basic properties of as broad a range of fundamental graph algorithms as possible.

You will most appreciate the material here if you have had a course covering basic principles of algorithm design and analysis and programming experience in a high-level language such as C, Java, or C++. *Algorithms in C, Third Edition, Parts 1–4* is certainly adequate preparation. This volume assumes basic knowledge about arrays, linked lists, and ADT design, and makes uses of priority-queue, symbol-table, and union-find ADTs—all of which are described in detail in Parts 1–4 (and in many other introductory texts on algorithms and data structures).

Basic properties of graphs and graph algorithms are developed from first principles, but full understanding of the properties of the algorithms can lead to deep and difficult mathematics. Although the discussion of advanced mathematical concepts is brief, general, and descriptive, you certainly need a higher level of mathematical maturity to appreciate graph algorithms than you do for the topics in Parts 1–4. Still, readers at various levels of mathematical maturity will be able to profit from this book. The topic dictates this approach: some elementary graph algorithms that should be understood and used by everyone differ only slightly from some advanced algorithms that are not understood by anyone. The primary intent here is to place important algorithms in context with other methods throughout the book, not to teach all of the mathematical material. But the rigorous treatment demanded by good mathematics often leads us to good programs, so I have tried to provide a balance between the formal treatment favored by theoreticians and the coverage needed by practitioners, without sacrificing rigor.

Use in the Curriculum

There is a great deal of flexibility in how the material here can be taught, depending on the taste of the instructor and the preparation of the students. The algorithms described have found widespread use for years, and represent an essential body of knowledge for both the practicing programmer and the computer science student. There is sufficient coverage of basic material for the book to be used in a course on data structures and algorithms, and there is sufficient detail and coverage of advanced material for the book to be used for a course on graph algorithms. Some instructors may wish to emphasize

implementations and practical concerns; others may wish to emphasize analysis and theoretical concepts.

For a more comprehensive course, this book is also available in a special bundle with Parts 1–4; thereby instructors can cover fundamentals, data structures, sorting, searching, and graph algorithms in one consistent style. A complete set of slide masters for use in lectures, sample programming assignments, interactive exercises for students, and other course materials may be found by accessing the book's home page.

The exercises—nearly all of which are new to this edition—fall into several types. Some are intended to test understanding of material in the text, and simply ask readers to work through an example or to apply concepts described in the text. Others involve implementing and putting together the algorithms, or running empirical studies to compare variants of the algorithms and to learn their properties. Still other exercises are a repository for important information at a level of detail that is not appropriate for the text. Reading and thinking about the exercises will pay dividends for every reader.

Algorithms of Practical Use

Anyone wanting to use a computer more effectively can use this book for reference or for self-study. People with programming experience can find information on specific topics throughout the book. To a large extent, you can read the individual chapters in the book independently of the others, although, in some cases, algorithms in one chapter make use of methods from a previous chapter.

The orientation of the book is to study algorithms likely to be of practical use. The book provides information about the tools of the trade to the point that readers can confidently implement, debug, and put to work algorithms to solve a problem or to provide functionality in an application. Full implementations of the methods discussed are included, as are descriptions of the operations of these programs on a consistent set of examples. Because we work with real code, rather than write pseudo-code, the programs can be put to practical use quickly. Program listings are available from the book's home page.

Indeed, one practical application of the algorithms has been to produce the hundreds of figures throughout the book. Many algo-

rithms are brought to light on an intuitive level through the visual dimension provided by these figures.

Characteristics of the algorithms and of the situations in which they might be useful are discussed in detail. Although not emphasized, connections to the analysis of algorithms and theoretical computer science are developed in context. When appropriate, empirical and analytic results are presented to illustrate why certain algorithms are preferred. When interesting, the relationship of the practical algorithms being discussed to purely theoretical results is described. Specific information on performance characteristics of algorithms and implementations is synthesized, encapsulated, and discussed throughout the book.

Programming Language

The programming language used for all of the implementations is C (versions of the book in C++ and Java are under development). Any particular language has advantages and disadvantages; we use C in this book because it is widely available and provides the features needed for the implementations here. The programs can be translated easily to other modern programming languages because relatively few constructs are unique to C. We use standard C idioms when appropriate, but this book is not intended to be a reference work on C programming.

We strive for elegant, compact, and portable implementations, but we take the point of view that efficiency matters, so we try to be aware of the code's performance characteristics at all stages of development. There are many new programs in this edition, and many of the old ones have been reworked, primarily to make them more readily useful as abstract-data-type implementations. Extensive comparative empirical tests on the programs are discussed throughout the book.

A goal of this book is to present the algorithms in as simple and direct a form as possible. The style is consistent whenever possible so that similar programs look similar. For many of the algorithms, the similarities remain regardless of which language is used: Dijkstra's algorithm (to pick one prominent example) is Dijkstra's algorithm, whether expressed in Algol-60, Basic, Fortran, Smalltalk, Ada, Pascal,

C, C++, Modula-3, PostScript, Java, or any of the countless other programming languages and environments in which it has proved to be an effective graph-processing method.

Acknowledgments

Many people gave me helpful feedback on earlier versions of this book. In particular, hundreds of students at Princeton and Brown have suffered through preliminary drafts over the years. Special thanks are due to Trina Avery and Tom Freeman for their help in producing the first edition; to Janet Incerpi for her creativity and ingenuity in persuading our early and primitive digital computerized typesetting hardware and software to produce the first edition; to Marc Brown for his part in the algorithm visualization research that was the genesis of so many of the figures in the book; to Dave Hanson for his willingness to answer all of my questions about C; and to Kevin Wayne, for patiently answering my basic questions about networks. I would also like to thank the many readers who have provided me with detailed comments about various editions, including Guy Almes, Jon Bentley, Marc Brown, Jay Gischer, Allan Heydon, Kennedy Lemke, Udi Manber, Dana Richards, John Reif, M. Rosenfeld, Stephen Seidman, Michael Quinn, and William Ward.

To produce this new edition, I have had the pleasure of working with Peter Gordon and Helen Goldstein at Addison-Wesley, who have patiently shepherded this project as it has evolved from a standard update to a massive rewrite. It has also been my pleasure to work with several other members of the professional staff at Addison-Wesley. The nature of this project made the book a somewhat unusual challenge for many of them, and I much appreciate their forbearance.

I have gained two new mentors in writing this book, and particularly want to express my appreciation to them. First, Steve Summit carefully checked early versions of the manuscript on a technical level, and provided me with literally thousands of detailed comments, particularly on the programs. Steve clearly understood my goal of providing elegant, efficient, and effective implementations, and his comments not only helped me to provide a measure of consistency across the implementations, but also helped me to improve many of them substantially. Second, Lyn Dupre also provided me with thousands of detailed com-

ments on the manuscript, which were invaluable in helping me not only to correct and avoid grammatical errors, but also—more important—to find a consistent and coherent writing style that helps bind together the daunting mass of technical material here. I am extremely grateful for the opportunity to learn from Steve and Lyn—their input was vital in the development of this book.

Much of what I have written here I have learned from the teaching and writings of Don Knuth, my advisor at Stanford. Although Don had no direct influence on this work, his presence may be felt in the book, for it was he who put the study of algorithms on the scientific footing that makes a work such as this possible. My friend and colleague Philippe Flajolet, who has been a major force in the development of the analysis of algorithms as a mature research area, has had a similar influence on this work.

I am deeply thankful for the support of Princeton University, Brown University, and the Institut National de Recherce en Informatique et Automatique (INRIA), where I did most of the work on the books; and of the Institute for Defense Analyses and the Xerox Palo Alto Research Center, where I did some work on the books while visiting. Many parts of these books are dependent on research that has been generously supported by the National Science Foundation and the Office of Naval Research. Finally, I thank Bill Bowen, Aaron Lemonick, and Neil Rudenstine for their support in building an academic environment at Princeton in which I was able to prepare this book, despite my numerous other responsibilities.

Robert Sedgewick
Marly-le-Roi, France, February, 1983
Princeton, New Jersey, January, 1990
Jamestown, Rhode Island, May, 2001

This page intentionally left blank

To Adam, Andrew, Brett, Robbie,
and especially Linda

Notes on Exercises

Classifying exercises is an activity fraught with peril, because readers of a book such as this come to the material with various levels of knowledge and experience. Nonetheless, guidance is appropriate, so many of the exercises carry one of four annotations, to help you decide how to approach them.

Exercises that *test your understanding* of the material are marked with an open triangle, as follows:

▷ **17.2** Consider the graph

```
3-7 1-4 7-8 0-5 5-2 3-8 2-9 0-6 4-9 2-6 6-4.
```

Draw the its DFS tree and use the tree to find the graph's bridges and edge-connected components.

Most often, such exercises relate directly to examples in the text. They should present no special difficulty, but working them might teach you a fact or concept that may have eluded you when you read the text.

Exercises that *add new and thought-provoking* information to the material are marked with an open circle, as follows:

○ **18.2** Write a program that counts the number of different possible results of topologically sorting a given DAG.

Such exercises encourage you to think about an important concept that is related to the material in the text, or to answer a question that may have occurred to you when you read the text. You may find it worthwhile to read these exercises, even if you do not have the time to work them through.

Exercises that are intended to *challenge you* are marked with a black dot, as follows:

• **19.2** Describe how you would find the MST of a graph so large that only V edges can fit into main memory at once.

Such exercises may require a substantial amount of time to complete, depending upon your experience. Generally, the most productive approach is to work on them in a few different sittings.

A few exercises that are *extremely difficult* (by comparison with most others) are marked with two black dots, as follows:

•• **20.2** Develop a reasonable generator for random graphs with V vertices and E edges such that the running time of the PFS implementation of Dijkstra's algorithm is nonlinear.

These exercises are similar to questions that might be addressed in the research literature, but the material in the book may prepare you to enjoy trying to solve them (and perhaps succeeding).

The annotations are intended to be neutral with respect to your programming and mathematical ability. Those exercises that require expertise in programming or in mathematical analysis are self-evident. All readers are encouraged to test their understanding of the algorithms by implementing them. Still, an exercise such as this one is straightforward for a practicing programmer or a student in a programming course, but may require substantial work for someone who has not recently programmed:

> • **17.2** Write a program that generates V random points in the plane, then builds a network with edges (in both directions) connecting all pairs of points within a given distance d of one another (see Program 3.20), setting each edge's weight to the distance between the two points that it connects. Determine how to set d so that the expected number of edges is E.

In a similar vein, all readers are encouraged to strive to appreciate the analytic underpinnings of our knowledge about properties of algorithms. Still, an exercise such as this one is straightforward for a scientist or a student in a discrete mathematics course, but may require substantial work for someone who has not recently done mathematical analysis:

> ○ **18.2** How many digraphs correspond to each undirected graph with V vertices and E edges?

There are far too many exercises for you to read and assimilate them all; my hope is that there are enough exercises here to stimulate you to strive to come to a broader understanding on the topics that interest you than you can glean by simply reading the text.

This page intentionally left blank

Contents

Graph Algorithms

Chapter 17. Graph Properties and Types **3**

17.1 Glossary · *7*

17.2 Graph ADT · *16*

17.3 Adjacency-Matrix Representation · *21*

17.4 Adjacency-Lists Representation · *27*

17.5 Variations, Extensions, and Costs · *31*

17.6 Graph Generators · *40*

17.7 Simple, Euler, and Hamilton Paths · *50*

17.8 Graph-Processing Problems · *64*

Chapter 18. Graph Search **75**

18.1 Exploring a Maze · *76*

18.2 Depth-First Search · *81*

18.3 Graph-Search ADT Functions · *86*

18.4 Properties of DFS Forests · *91*

18.5 DFS Algorithms · *99*

18.6 Separability and Biconnectivity · *106*

18.7 Breadth-First Search · *114*

18.8 Generalized Graph Search · *124*

18.9 Analysis of Graph Algorithms · *133*

Chapter 19. Digraphs and DAGs **141**

19.1 Glossary and Rules of the Game · *144*

19.2 Anatomy of DFS in Digraphs · *152*

19.3 Reachability and Transitive Closure · *161*

19.4 Equivalence Relations and Partial Orders · *174*

19.5 DAGs · *178*

19.6 Topological Sorting · *183*

19.7 Reachability in DAGs · *193*

19.8 Strong Components in Digraphs · *196*

19.9 Transitive Closure Revisited · *208*

19.10 Perspective · *212*

Chapter 20. Minimum Spanning Trees **219**

20.1 Representations · *222*

20.2 Underlying Principles of MST Algorithms · *228*

20.3 Prim's Algorithm and Priority-First Search · *235*

20.4 Kruskal's Algorithm · *246*

20.5 Boruvka's Algorithm · *252*

20.6 Comparisons and Improvements · *255*

20.7 Euclidean MST · *261*

Chapter 21. Shortest Paths **265**

21.1 Underlying Principles · *273*

21.2 Dijkstra's algorithm · *280*

21.3 All-Pairs Shortest Paths · *290*

21.4 Shortest Paths in Acyclic Networks · *300*

21.5 Euclidean Networks · *308*

21.6 Reduction · *314*

21.7 Negative Weights · *331*

21.8 Perspective · *350*

Chapter 22. Network Flows **353**

22.1 Flow Networks · *359*

22.2 Augmenting-Path Maxflow Algorithms · *370*

22.3 Preflow-Push Maxflow Algorithms · *396*

22.4 Maxflow Reductions · *411*

22.5 Mincost Flows · *429*

22.6 Network Simplex Algorithm · *439*

22.7 Mincost-Flow Reductions · *457*

22.8 Perspective · *467*

References for Part Five **473**

Index **475**

This page intentionally left blank

PART
FIVE

Graph Algorithms

This page intentionally left blank

CHAPTER SEVENTEEN

Graph Properties and Types

MANY COMPUTATIONAL APPLICATIONS naturally involve not just a set of *items*, but also a set of *connections* between pairs of those items. The relationships implied by these connections lead immediately to a host of natural questions: Is there a way to get from one item to another by following the connections? How many other items can be reached from a given item? What is the best way to get from this item to this other item?

To model such situations, we use abstract objects called *graphs*. In this chapter, we examine basic properties of graphs in detail, setting the stage for us to study a variety of algorithms that are useful for answering questions of the type just posed. These algorithms make effective use of many of the computational tools that we considered in Parts 1 through 4. They also serve as the basis for attacking problems in important applications whose solution we could not even contemplate without good algorithmic technology.

Graph theory, a major branch of combinatorial mathematics, has been studied intensively for hundreds of years. Many important and useful properties of graphs have been proved, yet many difficult problems remain unresolved. In this book, while recognizing that there is much still to be learned, we draw from this vast body of knowledge about graphs what we need to understand and use a broad variety of useful and fundamental algorithms.

Like so many of the other problem domains that we have studied, the algorithmic investigation of graphs is relatively recent. Although a few of the fundamental algorithms are old, the majority of the interesting ones have been discovered within the last few decades. Even

the simplest graph algorithms lead to useful computer programs, and the nontrivial algorithms that we examine are among the most elegant and interesting algorithms known.

To illustrate the diversity of applications that involve graph processing, we begin our exploration of algorithms in this fertile area by considering several examples.

Maps A person who is planning a trip may need to answer questions such as, "What is the *least expensive* way to get from Princeton to San Jose?" A person more interested in time than in money may need to know the answer to the question "What is the *fastest* way to get from Princeton to San Jose?" To answer such questions, we process information about *connections* (travel routes) between *items* (towns and cities).

Hypertexts When we browse the Web, we encounter documents that contain references (links) to other documents, and we move from document to document by clicking on the links. The entire web is a graph, where the items are documents and the connections are links. Graph-processing algorithms are essential components of the search engines that help us locate information on the web.

Circuits An electric circuit comprises elements such as transistors, resistors, and capacitors that are intricately wired together. We use computers to control machines that make circuits, and to check that the circuits perform desired functions. We need to answer simple questions such as, "Is a short-circuit present?" as well as complicated questions such as, "Can we lay out this circuit on a chip without making any wires cross?" In this case, the answer to the first question depends on only the properties of the connections (wires), whereas the answer to the second question requires detailed information about the wires, the items that those wires connect, and the physical constraints of the chip.

Schedules A manufacturing process requires a variety of tasks to be performed, under a set of constraints that specifies that certain tasks cannot be started until certain other tasks have been completed. We represent the constraints as connections between the tasks (items), and we are faced with a classical *scheduling* problem: How do we schedule the tasks such that we both respect the given constraints and complete the whole process in the least amount of time?

Transactions A telephone company maintains a database of telephone-call traffic. Here the connections represent telephone calls. We are interested in knowing about the nature of the interconnection structure because we want to lay wires and build switches that can handle the traffic efficiently. As another example, a financial institution tracks buy/sell orders in a market. A connection in this situation represents the transfer of cash between two customers. Knowledge of the nature of the connection structure in this instance may enhance our understanding of the nature of the market.

Matching Students apply for positions in selective institutions such as social clubs, universities, or medical schools. Items correspond to the students and the institutions; connections correspond to the applications. We want to discover methods for matching interested students with available positions.

Networks A computer network consists of interconnected sites that send, forward, and receive messages of various types. We are interested not just in knowing that it is possible to get a message from every site to every other site, but also in maintaining this connectivity for all pairs of sites as the network changes. For example, we might wish to check a given network to be sure that no small set of sites or connections is so critical that losing it would disconnect any remaining pair of sites.

Program structure A compiler builds graphs to represent the call structure of a large software system. The items are the various functions or modules that comprise the system; connections are associated either with the possibility that one function might call another (static analysis) or with actual calls while the system is in operation (dynamic analysis). We need to analyze the graph to determine how best to allocate resources to the program most efficiently .

These examples indicate the range of applications for which graphs are the appropriate abstraction, and also the range of computational problems that we might encounter when we work with graphs. Such problems will be our focus in this book. In many of these applications as they are encountered in practice, the volume of data involved is truly huge, and efficient algorithms make the difference between whether or not a solution is at all feasible.

We have already encountered graphs, briefly, in Part 1. Indeed, the first algorithms that we considered in detail, the union-find algo-

rithms in Chapter 1, are prime examples of graph algorithms. We also used graphs in Chapter 3 as an illustration of applications of two-dimensional arrays and linked lists, and in Chapter 5 to illustrate the relationship between recursive programs and fundamental data structures. Any linked data structure is a representation of a graph, and some familiar algorithms for processing trees and other linked structures are special cases of graph algorithms. The purpose of this chapter is to provide a context for developing an understanding of graph algorithms ranging from the simple ones in Part 1 to the sophisticated ones in Chapters 18 through 22.

As always, we are interested in knowing which are the most efficient algorithms that solve a particular problem. The study of the performance characteristics of graph algorithms is challenging because

- The cost of an algorithm depends not just on properties of the set of items, but also on numerous properties of the set of connections (and global properties of the graph that are implied by the connections).
- Accurate models of the types of graphs that we might face are difficult to develop.

We often work with worst-case performance bounds on graph algorithms, even though they may represent pessimistic estimates on actual performance in many instances. Fortunately, as we shall see, a number of algorithms are *optimal* and involve little unnecessary work. Other algorithms consume the same resources on all graphs of a given size. We can predict accurately how such algorithms will perform in specific situations. When we cannot make such accurate predictions, we need to pay particular attention to properties of the various types of graphs that we might expect in practical applications and must assess how these properties might affect the performance of our algorithms.

We begin by working through the basic definitions of graphs and the properties of graphs, examining the standard nomenclature that is used to describe them. Following that, we define the basic ADT (abstract data type) interfaces that we use to study graph algorithms and the two most important data structures for representing graphs—the *adjacency-matrix* representation and the *adjacency-lists* representation, and various approaches to implementing basic ADT functions. Then, we consider client programs that can generate random graphs, which we can use to test our algorithms and to learn

properties of graphs. All this material provides a basis for us to introduce graph-processing algorithms that solve three classical problems related to finding paths in graphs, which illustrate that the difficulty of graph problems can differ dramatically even when they might seem similar. We conclude the chapter with a review of the most important graph-processing problems that we consider in this book, placing them in context according to the difficulty of solving them.

17.1 Glossary

A substantial amount of nomenclature is associated with graphs. Most of the terms have straightforward definitions, and, for reference, it is convenient to consider them in one place: here. We have already used some of these concepts when considering basic algorithms in Part 1; others of them will not become relevant until we address associated advanced algorithms in Chapters 18 through 22.

Definition 17.1 *A* **graph** *is a set of* **vertices** *plus a set of* **edges** *that connect pairs of distinct vertices (with at most one edge connecting any pair of vertices).*

We use the names `0` through `V-1` for the vertices in a V-vertex graph. The main reason that we choose this system is that we can access quickly information corresponding to each vertex, using array indexing. In Section 17.6, we consider a program that uses a symbol table to establish a 1–1 mapping to associate V arbitrary vertex names with the V integers between 0 and $V - 1$. With that program in hand, we can use indices as vertex names (for notational convenience) without loss of generality. We sometimes assume that the set of vertices is defined implicitly, by taking the set of edges to define the graph and considering only those vertices that are included in at least one edge. To avoid cumbersome usage such as "the ten-vertex graph with the following set of edges," we do not explicitly mention the number of vertices when that number is clear from the context. By convention, we always denote the number of vertices in a given graph by V, and denote the number of edges by E.

We adopt as standard this definition of a graph (which we first encountered in Chapter 5), but note that it embodies two technical simplifications. First, it disallows duplicate edges (mathematicians

sometimes refer to such edges as *parallel* edges, and a graph that can contain them as a *multigraph*). Second, it disallows edges that connect vertices to themselves; such edges are called *self-loops*. Graphs that have no parallel edges or self-loops are sometimes referred to as *simple graphs*.

We use simple graphs in our formal definitions because it is easier to express their basic properties and because parallel edges and self-loops are not needed in many applications. For example, we can bound the number of edges in a simple graph with a given number of vertices.

Property 17.1 *A graph with V vertices has at most $V(V-1)/2$ edges.*

Proof: The total of V^2 possible pairs of vertices includes V self-loops and accounts twice for each edge between distinct vertices, so the number of edges is at most $(V^2 - V)/2 = V(V-1)/2$. ■

No such bound holds if we allow parallel edges: a graph that is not simple might consist of two vertices and billions of edges connecting them (or even a single vertex and billions of self-loops).

For some applications, we might consider the elimination of parallel edges and self-loops to be a data-processing problem that our implementations must address. For other applications, ensuring that a given set of edges represents a simple graph may not be worth the trouble. Throughout the book, whenever it is more convenient to address an application or to develop an algorithm by using an extended definition that includes parallel edges or self-loops, we shall do so. For example, self-loops play a critical role in a classical algorithm that we will examine in Section 17.4; and parallel edges are common in the applications that we address in Chapter 22. Generally, it is clear from the context whether we intend the term "graph" to mean "simple graph" or "multigraph" or "multigraph with self-loops."

Mathematicians use the words *vertex* and *node* interchangeably, but we generally use *vertex* when discussing graphs and *node* when discussing representations—for example, in C data structures. We normally assume that a vertex can have a name and can carry other associated information. Similarly, the words *arc, edge,* and *link* are all widely used by mathematicians to describe the abstraction embodying a connection between two vertices, but we consistently use *edge* when discussing graphs and *link* when discussing C data structures.

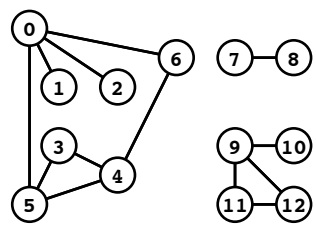
When there is an edge connecting two vertices, we say that the vertices are *adjacent to* one another and that the edge is *incident on* both vertices. The *degree* of a vertex is the number of edges incident on it. We use the notation `v-w` to represent an edge that connects `v` and `w`; the notation `w-v` is an alternative way to represent the same edge.

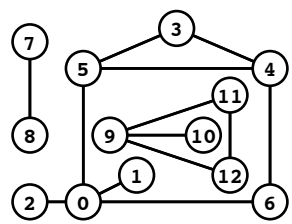
A *subgraph* is a subset of a graph's edges (and associated vertices) that constitutes a graph. Many computational tasks involve identifying subgraphs of various types. If we identify a subset of a graph's vertices, we call that subset, together with all edges that connect two of its members, the *induced subgraph* associated with those vertices.

We can draw a graph by marking points for the vertices and drawing lines connecting them for the edges. A drawing gives us intuition about the structure of the graph; but this intuition can be misleading, because the graph is defined independently of the representation. For example, the two drawings in Figure 17.1 and the list of edges represent the same graph, because the graph is only its (unordered) set of vertices and its (unordered) set of edges (pairs of vertices)—nothing more. Although it suffices to consider a graph simply as a set of edges, we examine other representations that are particularly suitable as the basis for graph data structures in Section 17.4.

Placing the vertices of a given graph on the plane and drawing them and the edges that connect them is known as *graph drawing*. The possible vertex placements, edge-drawing styles, and aesthetic constraints on the drawing are limitless. Graph-drawing algorithms that respect various natural constraints have been studied heavily and have many successful applications (*see reference section*). For example, one of the simplest constraints is to insist that edges do not intersect. A *planar graph* is one that can be drawn in the plane without any edges crossing. Determining whether or not a graph is planar is a fascinating algorithmic problem that we discuss briefly in Section 17.8. Being able to produce a helpful visual representation is a useful goal, and graph drawing is a fascinating field of study, but successful drawings are often difficult to realize. Many graphs that have huge numbers of vertices and edges are abstract objects for which no suitable drawing is feasible.

For some applications, such as graphs that represent maps or circuits, a graph drawing can carry considerable information because






```
0-5      5-4     7-8
4-3      0-2     9-11
0-1     11-12    5-3
9-12     9-10
6-4      0-6
```

Figure 17.1
Three different representations of the same graph

A graph is defined by its vertices and its edges, not by the way that we choose to draw it. These two drawings depict the same graph, as does the list of edges (bottom)*, given the additional information that the graph has 13 vertices labeled* `0` *through* `12`.

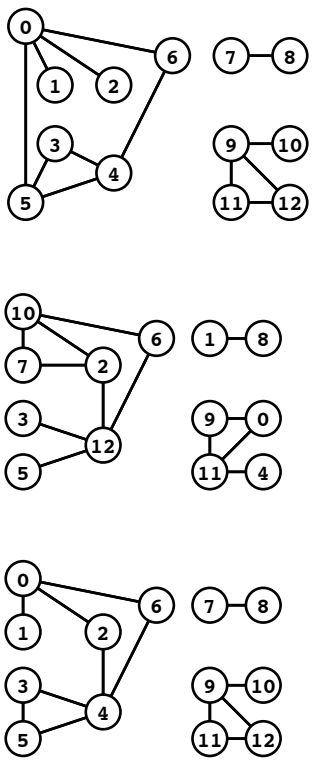


Figure 17.2
Graph isomorphism examples

The top two graphs are isomorphic because we can relabel the vertices to make the two sets of edges identical (to make the middle graph the same as the top graph, change 10 *to* 4, 7 *to* 3, 2 *to* 5, 3 *to* 1, 12 *to* 0, 5 *to* 2, 9 *to* 11, 0 *to* 12, 11 *to* 9, 1 *to* 7, *and* 4 *to* 10*). The bottom graph is not isomorphic to the others because there is no way to relabel the vertices to make its set of edges identical to either.*

the vertices correspond to points in the plane and the distances between them are relevant. We refer to such graphs as *Euclidean graphs*. For many other applications, such as graphs that represent relationships or schedules, the graphs simply embody connectivity information, and no particular geometric placement of vertices is ever implied. We consider examples of algorithms that exploit the geometric information in Euclidean graphs in Chapters 20 and 21, but we primarily work with algorithms that make no use of any geometric information, and stress that graphs are generally independent of any particular representation in a drawing or in a computer.

Focusing solely on the connections themselves, we might wish to view the vertex labels as merely a notational convenience, and to regard two graphs as being the same if they differ in only the vertex labels. Two graphs are *isomorphic* if we can change the vertex labels on one to make its set of edges identical to the other. Determining whether or not two graphs are isomorphic is a difficult computational problem (see Figure 17.2 and Exercise 17.5). It is challenging because there are $V!$ possible ways to label the vertices—far too many for us to try all the possibilities. Therefore, despite the potential appeal of reducing the number of different graph structures that we have to consider by treating isomorphic graphs as identical structures, we rarely do so.

As we saw with trees in Chapter 5, we are often interested in basic structural properties that we can deduce by considering specific sequences of edges in a graph.

Definition 17.2 *A* **path** *in a graph is a sequence of vertices in which each successive vertex (after the first) is adjacent to its predecessor in the path. In a* **simple path**, *the vertices and edges are distinct. A* **cycle** *is a path that is simple except that the first and final vertices are the same.*

We sometimes use the term *cyclic path* to refer to a path whose first and final vertices are the same (and is otherwise not necessarily simple); and we use the term *tour* to refer to a cyclic path that includes every vertex. An equivalent way to define a path is as the sequence of *edges* that connect the successive vertices. We emphasize this in our notation by connecting vertex names in a path in the same way as we connect them in an edge. For example, the simple paths in Figure 17.1 include 3-4-6-0-2, and 9-12-11, and the cycles in the graph include

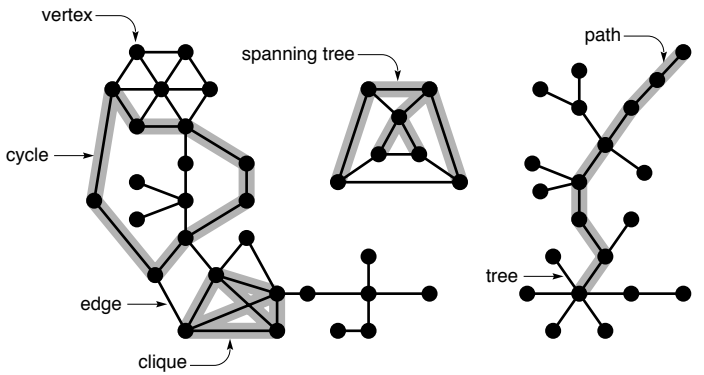


Figure 17.3
Graph terminology

This graph has 55 vertices, 70 edges, and 3 connected components. One of the connected components is a tree (right). *The graph has many cycles, one of which is highlighted in the large connected component* (left). *The diagram also depicts a spanning tree in the small connected component* (center). *The graph as a whole does not have a spanning tree, because it is not connected.*

`0-6-4-3-5-0` and `5-4-3-5`. We define the *length* of a path or a cycle to be its number of edges.

We adopt the convention that each single vertex is a path of length 0 (a path from the vertex to itself with no edges on it, which is different from a self-loop). Apart from this convention, in a graph with no parallel edges and no self-loops, a pair of vertices uniquely determines an edge, paths must have on them at least two distinct vertices, and cycles must have on them at least three distinct edges and three distinct vertices.

We say that two simple paths are *disjoint* if they have no vertices in common other than, possibly, their endpoints. Placing this condition is slightly weaker than insisting that the paths have no vertices at all in common, and is useful because we can combine simple disjoint paths from s to t and t to u to get a simple disjoint path from s to u if s and u are different (and to get a cycle if s and u are the same). The term *vertex disjoint* is sometimes used to distinguish this condition from the stronger condition of *edge disjoint*, where we require that the paths have no *edge* in common.

Definition 17.3 *A graph is a* **connected graph** *if there is a path from every vertex to every other vertex in the graph. A graph that is not connected consists of a set of* **connected components**, *which are maximal connected subgraphs.*

The term *maximal connected subgraph* means that there is no path from a subgraph vertex to any vertex in the graph that is not in the subgraph. Intuitively, if the vertices were physical objects, such as

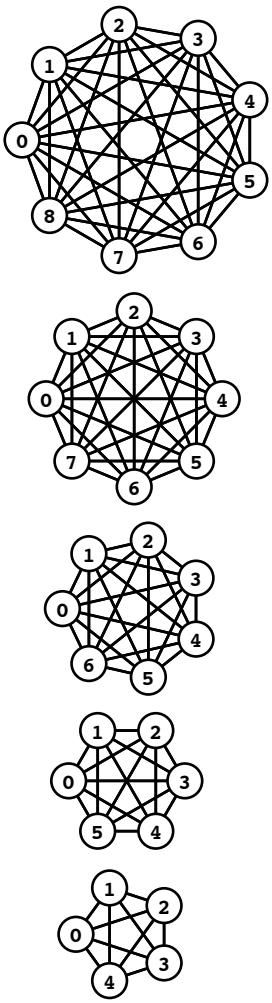


Figure 17.4
Complete graphs

These complete graphs, with every vertex connected to every other vertex, have 10, 15, 21, 28, and 36 edges (bottom to top). *Every graph with between 5 and 9 vertices (there are more than 68 billion such graphs) is a subgraph of one of these graphs.*

knots or beads, and the edges were physical connections, such as strings or wires, a connected graph would stay in one piece if picked up by any vertex, and a graph that is not connected comprises two or more such pieces.

Definition 17.4 *An acyclic connected graph is called a* **tree** *(see Chapter 4). A set of trees is called a* **forest**. *A* **spanning tree** *of a connected graph is a subgraph that contains all of that graph's vertices and is a single tree. A* **spanning forest** *of a graph is a subgraph that contains all of that graph's vertices and is a forest.*

For example, the graph illustrated in Figure 17.1 has three connected components, and is spanned by the forest `7-8 9-10 9-11 9-12 0-1 0-2 0-5 5-3 5-4 4-6` (there are many other spanning forests). Figure 17.3 highlights these and other features in a larger graph.

We explore further details about trees in Chapter 4, and look at various equivalent definitions. For example, a graph G with V vertices is a tree if and only if it satisfies *any* of the following four conditions:

- G has $V-1$ edges and no cycles.
- G has $V-1$ edges and is connected.
- Exactly one simple path connects each pair of vertices in G.
- G is connected, but removing any edge disconnects it.

Any one of these conditions is necessary and sufficient to prove the other three, and we can develop other combinations of facts about trees from them (see Exercise 17.1). Formally, we should choose one condition to serve as a definition; informally, we let them collectively serve as the definition, and freely engage in usage such as the "acyclic connected graph" choice in Definition 17.4.

Graphs with all edges present are called *complete graphs* (see Figure 17.4). We define the *complement* of a graph G by starting with a complete graph that has the same set of vertices as the original graph, and removing the edges of G. The *union* of two graphs is the graph induced by the union of their sets of edges. The union of a graph and its complement is a complete graph. All graphs that have V vertices are subgraphs of the complete graph that has V vertices. The total number of different graphs that have V vertices is $2^{V(V-1)/2}$ (the number of different ways to choose a subset from the $V(V-1)/2$ possible edges). A complete subgraph is called a *clique*.

Most graphs that we encounter in practice have relatively few of the possible edges present. To quantify this concept, we define the

density of a graph to be the average vertex degree, or $2E/V$. A *dense graph* is a graph whose average vertex degree is proportional to V; a *sparse graph* is a graph whose complement is dense. In other words, we consider a graph to be dense if E is proportional to V^2 and sparse otherwise. This asymptotic definition is not necessarily meaningful for a particular graph, but the distinction is generally clear: A graph that has millions of vertices and tens of millions of edges is certainly sparse, and a graph that has thousands of vertices and millions of edges is certainly dense. We might contemplate processing a sparse graph with billions of vertices, but a dense graph with billions of vertices would have an overwhelming number of edges.

Knowing whether a graph is sparse or dense is generally a key factor in selecting an efficient algorithm to process the graph. For example, for a given problem, we might develop one algorithm that takes about V^2 steps and another that takes about $E \lg E$ steps. These formulas tell us that the second algorithm would be better for sparse graphs, whereas the first would be preferred for dense graphs. For example, a dense graph with millions of edges might have only thousands of vertices: in this case V^2 and E would be comparable in value, and the V^2 algorithm would be 20 times faster than the $E \lg E$ algorithm. On the other hand, a sparse graph with millions of edges also has millions of vertices, so the $E \lg E$ algorithm could be *millions* of times faster than the V^2 algorithm. We could make specific tradeoffs on the basis of analyzing these formulas in more detail, but it generally suffices in practice to use the terms *sparse* and *dense* informally to help us understand fundamental performance characteristics.

When analyzing graph algorithms, we assume that V/E is bounded by above a small constant, so that we can abbreviate expressions such as $V(V + E)$ to VE. This assumption comes into play only when the number of edges is tiny in comparison to the number of vertices—a rare situation. Typically, the number of edges far exceeds the number of vertices (V/E is much less than 1).

A *bipartite graph* is a graph whose vertices we can divide into two sets such that all edges connect a vertex in one set with a vertex in the other set. Figure 17.5 gives an example of a bipartite graph. Bipartite graphs arise in a natural way in many situations, such as the matching problems described at the beginning of this chapter. Any subgraph of a bipartite graph is bipartite.

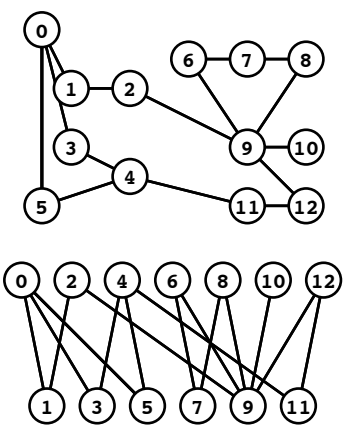


Figure 17.5
A bipartite graph

All edges in this graph connect odd-numbered vertices with even-numbered ones, so it is bipartite. The bottom diagram makes the property obvious.

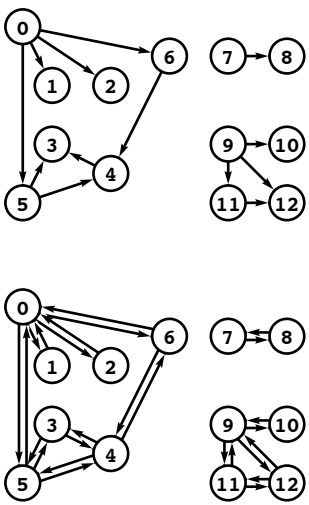


Figure 17.6
Two digraphs

The drawing at the top is a representation of the example graph in Figure 17.1 interpreted as a directed graph, where we take the edges to be ordered pairs and represent them by drawing an arrow from the first vertex to the second. It is also a DAG. The drawing at the bottom is a representation of the undirected graph from Figure 17.1 that indicates the way that we usually represent undirected graphs: as digraphs with two edges corresponding to each connection (one in each direction).

Graphs as defined to this point are called *undirected graphs*. In *directed graphs*, also known as *digraphs*, edges are one-way: we consider the pair of vertices that defines each edge to be an *ordered* pair that specifies a one-way adjacency where we think about having the ability to get from the first vertex to the second but not from the second vertex to the first. Many applications (for example, graphs that represent the web, scheduling constraints or telephone-call transactions) are naturally expressed in terms of digraphs.

We refer to edges in digraphs as *directed edges*, though that distinction is generally obvious in context (some authors reserve the term *arc* for directed edges). The first vertex in a directed edge is called the *source*; the second vertex is called the *destination*. (Some authors use the terms *head* and *tail*, respectively, to distinguish the vertices in directed edges, but we avoid this usage because of overlap with our use of the same terms in data-structure implementations.) We draw directed edges as arrows pointing from source to destination, and often say that the edge *points to* the destination. When we use the notation `v-w` in a digraph, we mean it to represent an edge that points from `v` to `w`; it is different from `w-v`, which represents an edge that points from `w` to `v`. We speak of the *indegree* and *outdegree* of a vertex (the number of edges where it is the destination and the number of edges where it is the source, respectively).

Sometimes, we are justified in thinking of an undirected graph as a digraph that has two directed edges (one in each direction); other times, it is useful to think of undirected graphs simply in terms of connections. Normally, as discussed in detail in Section 17.4, we use the same representation for directed and undirected graphs (see Figure 17.6). That is, we generally maintain two representations of each edge for undirected graphs, one pointing in each direction, so that we can immediately answer questions such as, "Which vertices are connected to vertex v?"

Chapter 19 is devoted to exploring the structural properties of digraphs; they are generally more complicated than the corresponding properties for undirected graphs. A *directed cycle* in a digraph is a cycle in which all adjacent vertex pairs appear in the order indicated by (directed) graph edges. A *directed acyclic graph (DAG)*, is a digraph that has no directed cycles. A DAG (an acyclic digraph) is not the same as a tree (an acyclic undirected graph). Occasionally, we refer to

the *underlying undirected graph* of a digraph, meaning the undirected graph defined by the same set of edges, but where these edges are not interpreted as directed.

Chapters 20 through 22 are generally concerned with algorithms for solving various computational problems associated with graphs in which other information is associated with the vertices and edges. In *weighted graphs*, we associate numbers (*weights*) with each edge, which generally represents a distance or cost. We also might associate a weight with each vertex, or multiple weights with each vertex and edge. In Chapter 20 we work with weighted undirected graphs; in Chapters 21 and 22 we study weighted digraphs, which we also refer to as *networks*. The algorithms in Chapter 22 solve classic problems that arise from a particular interpretation of networks known as *flow networks*.

As was evident even in Chapter 1, the combinatorial structure of graphs is extensive. This extent of this structure is all the more remarkable because it springs forth from a simple mathematical abstraction. This underlying simplicity will be reflected in much of the code that we develop for basic graph processing. However, this simplicity sometimes masks complicated dynamic properties that require deep understanding of the combinatorial properties of graphs themselves. It is often far more difficult to convince ourselves that a graph algorithm works as intended than the compact nature of the code might suggest.

Exercises

17.1 Prove that any acyclic connected graph that has V vertices has $V - 1$ edges.

▷ **17.2** Give all the connected subgraphs of the graph

```
0-1 0-2 0-3 1-3 2-3.
```

▷ **17.3** Write down a list of the nonisomorphic cycles of the graph in Figure 17.1. For example, if your list contains `3-4-5-3`, it should not contain `3-5-4-3`, `4-5-3-4`, `4-3-5-4`, `5-3-4-5`, or `5-4-3-5`.

17.4 Consider the graph

```
3-7 1-4 7-8 0-5 5-2 3-8 2-9 0-6 4-9 2-6 6-4.
```

Determine the number of connected components, give a spanning forest, list all the simple paths with at least three vertices, and list all the nonisomorphic cycles (see Exercise 17.3).

○ **17.5** Consider the graphs defined by the following four sets of edges:

```
0-1 0-2 0-3 1-3 1-4 2-5 2-9 3-6 4-7 4-8 5-8 5-9 6-7 6-9 7-8
0-1 0-2 0-3 0-3 1-4 2-5 2-9 3-6 4-7 4-8 5-8 5-9 6-7 6-9 7-8
0-1 1-2 1-3 0-3 0-4 2-5 2-9 3-6 4-7 4-8 5-8 5-9 6-7 6-9 7-8
4-1 7-9 6-2 7-3 5-0 0-2 0-8 1-6 3-9 6-3 2-8 1-5 9-8 4-5 4-7
```

Which of these graphs are isomorphic to one another? Which of them are planar?

17.6 Consider the more than 68 billion graphs referred to in the caption to Figure 17.4. What percentage of them has fewer than nine vertices?

▷ **17.7** How many different subgraphs are there in a given graph with V vertices and E edges?

• **17.8** Give tight upper and lower bounds on the number of connected components in graphs that have V vertices and E edges.

○ **17.9** How many different undirected graphs are there that have V vertices and E edges?

••• **17.10** If we consider two graphs to be different only if they are not isomorphic, how many different graphs are there that have V vertices and E edges?

17.11 How many V-vertex graphs are bipartite?

17.2 Graph ADT

We develop our graph-processing algorithms within the context of an ADT that defines the tasks of interest, using the standard mechanisms considered in Chapter 4. Program 17.1 is the nucleus of the ADT interface that we use for this purpose. Basic graph representations and implementations for this ADT are the topic of Sections 17.3 through 17.5. Later in the book, whenever we consider a new graph-processing problem, we consider the algorithms that solve it and their implementations in the context of new ADT functions for this interface. This scheme allows us to address graph-processing tasks ranging from elementary maintenance functions to sophisticated solutions of difficult problems.

The interface is based on our standard mechanism that hides representations and implementations from client programs (see Section 4.8). It also includes a simple structure type definition that allows our programs to manipulate edges in a uniform way. The interface provides the basic mechanisms that allows clients to build graphs (by initializing the graph and then adding the edges), to maintain the graphs

Program 17.1 Graph ADT interface

This interface is a starting point for implementing and testing graph algorithms. Throughout this and the next several chapters, we shall add functions to this interface for solving various graph-processing problems. Various assumptions that simplify the code and other issues surrounding the design of a general-purpose graph-processing interface are discussed in the text.

The interface defines two data types: a simple `Edge` data type, including a constructor function `EDGE` that makes an `Edge` from two vertices; and a `Graph` data type, which is defined with the standard representation-independent construction from Chapter 4. The basic operations that we use to process graphs are ADT functions to create, copy, and destroy them; to add and delete edges; and to extract an edge list.

```
typedef struct { int v; int w; } Edge;
Edge EDGE(int, int);

typedef struct graph *Graph;
Graph GRAPHinit(int);
 void GRAPHinsertE(Graph, Edge);
 void GRAPHremoveE(Graph, Edge);
  int GRAPHedges(Edge [], Graph G);
Graph GRAPHcopy(Graph);
 void GRAPHdestroy(Graph);
```

(by removing some edges and adding others), and to retrieve the graphs (in the form of an array of edges).

The ADT in Program 17.1 is primarily a vehicle to allow us to develop and test algorithms; it is not a general-purpose interface. As usual, we work with the simplest interface that supports the basic graph-processing operations that we wish to consider. Defining such an interface for use in practical applications involves making numerous tradeoffs among simplicity, efficiency, and generality. We consider a few of these tradeoffs next; we address many others in the context of implementations and applications throughout this book.

We assume for simplicity that graph representations include integers `V` and `E` that contain the number of vertices and edges, respectively, so that we can refer directly to those values by name in ADT implementations. When convenient, we make other, similar, assumptions about

variables in graph representations, primarily to keep implementations compact. For convenience, we also provide the maximum possible number of vertices in the graph as an argument to the `GRAPHinit` ADT function so that implementations can allocate memory accordingly. We adopt these conventions solely to make the code compact and readable.

A slightly more general interface might provide the capability to add and remove vertices as well as edges (and might include functions that return the number of vertices and edges), making no assumptions about implementations. This design would allow for ADT implementations that grow and shrink their data structures as the graph grows and shrinks. We might also choose to work at an intermediate level of abstraction, and consider the design of interfaces that support higher-level abstract operations on graphs that we can use in implementations. We revisit this idea briefly in Section 17.5, after we consider several concrete representations and implementations.

A general graph ADT needs to take into account parallel edges and self-loops, because nothing prevents a client program from calling `GRAPHinsertE` with an edge that is already present in the graph (parallel edge) or with an edge whose two vertex indices are the same (self-loop). It might be necessary to disallow such edges in some applications, desirable to include them in other applications, and possible to ignore them in still other applications. Self-loops are trivial to handle, but parallel edges can be costly to handle, depending on the graph representation. In certain situations, adding a *remove parallel edges* ADT function might be appropriate; then, implementations can let parallel edges collect, and clients can remove or otherwise process parallel edges when warranted.

Program 17.1 includes a function for implementations to return a graph's set of edges to a client, in an array. A graph is nothing more nor less than its set of edges, and we often need a way to retrieve a graph in this form, regardless of its internal representation. We might even consider an *array of edges* representation as the basis for an ADT implementation (see Exercise 17.15). That representation, however, does not provide the flexibility that we need to perform efficiently the basic graph-processing operations that we shall be studying.

In this book, we generally work with *static* graphs, which have a fixed number of vertices V and edges E. Generally, we build the

graphs by executing E calls to `GRAPHinsertE`, then process them by calling some ADT function that takes a graph as argument and returns some information about that graph. *Dynamic* problems, where we intermix graph processing with edge and vertex insertion and removal, take us into the realm of *online algorithms* (also known as *dynamic algorithms*), which present a different set of challenges. For example, the union-find problem that we considered in Chapter 1 is an example of an online algorithm, because we can get information about the connectivity of a graph as we insert edges. The ADT in Program 17.1 supports *insert edge* and *remove edge* operations, so clients are free to use them to make changes in graphs, but there may be performance penalties for certain sequences of operations. For example, union-find algorithms are effective for only those clients that do not use *remove edge*.

The ADT might also include a function that takes an array of edges as an argument for use in initializing the graph. We could easily implement this function by calling `GRAPHinsert` for each of the edges (see Exercise 17.13) or, depending on the graph representation, we might be able to craft a more efficient implementation.

We might also provide graph-traversal functions that call client-supplied functions for each edge or each vertex in the graph. For some simple problems, using the array returned by `GRAPHedges` might suffice. Most of our implementations, however, do more complicated traversals that reveal information about the graph's structure, while implementing functions that provide a higher level of abstraction to clients.

In Sections 17.3 through 17.5, we examine the primary classical graph representations and implementations of the ADT functions in Program 17.1. These implementations provide a basis for us to expand the interface to include the graph-processing tasks that are our focus for the next several chapters.

When we consider a new graph-processing problem, we extend the ADT as appropriate to encompass functions that implement algorithms of interest for solving the problem. Generally these tasks fall into one of two broad categories:

- Compute the value of some measure of the graph.
- Compute some subset of the edges of the graph.

Program 17.2 Example of a graph-processing client

This program takes `V` and `E` from standard input, generates a random graph with `V` vertices and `E` edges, prints the graph if it is small, and computes (and prints) the number of connected components. It uses the ADT functions `GRAPHrand` (see Program 17.8), `GRAPHshow` (see Exercise 17.16 and Program 17.5), and `GRAPHcc` (see Program 18.4)).

```
#include <stdio.h>
#include "GRAPH.h"
main(int argc, char *argv[])
  { int V = atoi(argv[1]), E = atoi(argv[2]);
    Graph G = GRAPHrand(V, E);
    if (V < 20)
         GRAPHshow(G);
    else printf("%d vertices, %d edges, ", V, E);
    printf("%d component(s)\n", GRAPHcc(G));
  }
```

Examples of the former are the number of connected components and the length of the shortest path between two given vertices in the graph; examples of the latter are a spanning tree and the longest cycle containing a given vertex. Indeed, the terms that we defined in Section 17.1 immediately bring to mind a host of computational problems.

Program 17.2 is an example of a graph-processing client program. It uses the basic ADT of Program 17.1, augmented by a *generate random graph* ADT function that returns a random graph that contains a given number of vertices and edges (see Section 17.6), and a *connected components* ADT function that returns the number of connected components in a given graph (see Section 18.4). We use similar but more sophisticated clients to generate other types of graphs, to test algorithms, and to learn properties of graphs. The basic interface is amenable for use in any graph-processing application.

The first decision that we face in developing an ADT implementation is which graph representation to use. We have three basic requirements. First, we must be able to accommodate the types of graphs that we are likely to encounter in applications (and we also would prefer not to waste space). Second, we should be able to construct the requisite data structures efficiently. Third, we want to develop

efficient algorithms to solve our graph-processing problems without being unduly hampered by any restrictions imposed by the representation. Such requirements are standard ones for any domain that we consider—we emphasize them again them here because, as we shall see, different representations give rise to huge performance differences for even the simplest of problems.

Most graph-processing applications can be handled reasonably with one of two straightforward classical representations that are only slightly more complicated than the array-of-edges representation: the *adjacency-matrix* or the *adjacency-lists* representation. These representations, which we consider in detail in Sections 17.3 and 17.4, are based on elementary data structures (indeed, we discussed them both in Chapters 3 and 5 as example applications of sequential and linked allocation). The choice between the two depends primarily on whether the graph is dense or sparse, although, as usual, the nature of the operations to be performed also plays an important role in the decision on which to use.

Exercises

▷ **17.12** Write a program that builds a graph by reading edges (pairs of integers between 0 and $V - 1$) from standard input.

17.13 Write a representation-independent graph-initialization ADT function that, given an array of edges, returns a graph.

17.14 Write a representation-independent graph ADT function that uses `GRAPHedges` to print out all the edges in the graph, in the format used in this text (vertex numbers separated by a hyphen).

17.15 Provide an implementation of the ADT functions in Program 17.1 that uses an array of edges to represent the graph. Modify `GRAPHinit` to take the maximum number of edges allowed as its second argument, for use in allocating the edge array. Use a brute-force implementation of `GRAPHremoveE` that removes an edge `v-w` by scanning the array to find `v-w` or `w-v`, and then exchanges the edge found with the final one in the array. Disallow parallel edges by doing a similar scan in `GRAPHinsertE`.

17.3 Adjacency-Matrix Representation

An *adjacency-matrix* representation of a graph is a V-by-V array of Boolean values, with the entry in row v and column w defined to be 1 if there is an edge connecting vertex v and vertex w in the graph, and to

	0	1	2	3	4	5	6	7	8	9	10	11	12
0	0	1	1	0	0	1	1	0	0	0	0	0	0
1	1	0	0	0	0	0	0	0	0	0	0	0	0
2	1	0	0	0	0	0	0	0	0	0	0	0	0
3	0	0	0	0	1	1	0	0	0	0	0	0	0
4	0	0	0	1	0	1	1	0	0	0	0	0	0
5	1	0	0	1	1	0	0	0	0	0	0	0	0
6	1	0	0	0	1	0	0	0	0	0	0	0	0
7	0	0	0	0	0	0	0	0	1	0	0	0	0
8	0	0	0	0	0	0	0	1	0	0	0	0	0
9	0	0	0	0	0	0	0	0	0	0	1	1	1
10	0	0	0	0	0	0	0	0	0	1	0	0	0
11	0	0	0	0	0	0	0	0	0	1	0	0	1
12	0	0	0	0	0	0	0	0	0	1	0	1	0

Figure 17.7 Adjacency-matrix graph representation

This matrix is another representation of the graph depicted in Figure 17.1. It has a 1 in row v and column w whenever there is an edge connecting vertex v and vertex w. The array is symmetric about the diagonal. For example, the sixth row (and the sixth column) says that vertex `6` *is connected to vertices* `0` *and* `4`*. For some applications, we will adopt the convention that each vertex is connected to itself, and assign 1s on the main diagonal. The large blocks of 0s in the upper right and lower left corners are artifacts of the way we assigned vertex numbers for this example, not characteristic of the graph (except that they do indicate the graph to be sparse).*

Program 17.3 Graph ADT implementation (adjacency matrix)

This implementation of the interface in Program 17.1 uses a two-dimensional array. An implementation of the function `MATRIXint`, which allocates memory for the array and initializes it, is given in Program 17.4. The rest of the code is straightforward: An edge `i-j` is present in the graph if and only if `a[i][j]` and `a[j][i]` are both `1`. Edges are inserted and removed in constant time, and duplicate edges are silently ignored. Initialization and extracting all edges each take time proportional to V^2.

```
#include <stdlib.h>
#include "GRAPH.h"
struct graph { int V; int E; int **adj; };
Graph GRAPHinit(int V)
  { Graph G = malloc(sizeof *G);
    G->V = V; G->E = 0;
    G->adj = MATRIXint(V, V, 0);
    return G;
  }
void GRAPHinsertE(Graph G, Edge e)
  { int v = e.v, w = e.w;
    if (G->adj[v][w] == 0) G->E++;
    G->adj[v][w] = 1;
    G->adj[w][v] = 1;
  }
void GRAPHremoveE(Graph G, Edge e)
  { int v = e.v, w = e.w;
    if (G->adj[v][w] == 1) G->E--;
    G->adj[v][w] = 0;
    G->adj[w][v] = 0;
  }
int GRAPHedges(Edge a[], Graph G)
  { int v, w, E = 0;
    for (v = 0; v < G->V; v++)
      for (w = v+1; w < G->V; w++)
        if (G->adj[v][w] == 1)
          a[E++] = EDGE(v, w);
    return E;
  }
```

Program 17.4 Adjacency-matrix allocation and initialization

This program uses the standard C array-of-arrays representation for the two-dimensional adjacency matrix (see Section 3.7). It allocates `r` rows with `c` integers each, then initializes all entries to the value `val`. The call `MATRIXint(V, V, 0)` in Program 17.3 takes time proportional to V^2 to create a matrix that represents a V-vertex graph with no edges. For small V, the cost of V calls to `malloc` might predominate.

```
int **MATRIXint(int r, int c, int val)
  { int i, j;
    int **t = malloc(r * sizeof(int *));
    for (i = 0; i < r; i++)
      t[i] = malloc(c * sizeof(int));
    for (i = 0; i < r; i++)
      for (j = 0; j < c; j++)
        t[i][j] = val;
    return t;
  }
```

be 0 otherwise. Program 17.3 is an implementation of the graph ADT that uses a direct representation of this matrix. The implementation maintains a two-dimensional array of integers with the entry `a[v][w]` set to 1 if there is an edge connecting `v` and `w` in the graph, and set to 0 otherwise. In an undirected graph, each edge is actually represented by *two* entries: the edge `v-w` is represented by 1 values in *both* `a[v][w]` and `a[w][v]`, as is the edge `w-v`.

As mentioned in Section 17.2, we generally assume that the number of vertices is known to the client when the graph is initialized. For many applications, we might set the number of vertices as a compile-time constant and use statically allocated arrays, but Program 17.3 takes the slightly more general approach of allocating dynamically the space for the adjacency matrix. Program 17.4 is an implementation of the standard method of dynamically allocating a two-dimensional array in C, as an array of pointers, as depicted in Figure 17.8. Program 17.4 also includes code that initializes the graph by setting the array entries all to a given value. This operation takes time proportional to V^2. Error checks for insufficient memory are not included in

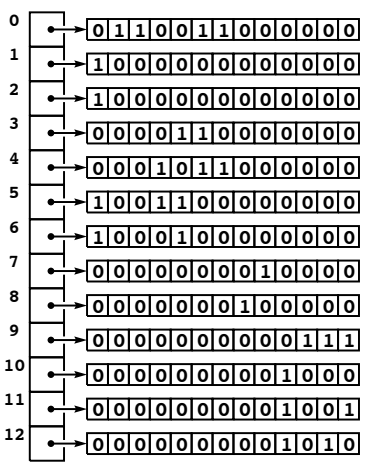


Figure 17.8 Adjacency matrix data structure

This figure depicts the C representation of the graph in Figure 17.1, as an array of arrays.

Program 17.4 for brevity—it is prudent programming practice to add them before using this code (see Exercise 17.22).

To add an edge, we set the two indicated array entries to 1. We do not allow parallel edges in this representation: If an edge is to be inserted for which the array entries are already 1, the code has no effect. In some ADT designs, it might be preferable to inform the client of the attempt to insert a parallel edge, perhaps using a return code from `GRAPHinsertE`. We do allow self-loops in this representation: An edge `v-v` is represented by a nonzero entry in `a[v][v]`.

To remove an edge, we set the two indicated array entries to 0. If a nonexistent edge (one for which the array entries are already 0) is to be removed, the code has no effect. Again, in some ADT designs, we might wish to arrange to inform the client of such conditions.

If we are processing huge graphs or huge numbers of small graphs, or space is otherwise tight, there are several ways to save space. For example, adjacency matrices that represent undirected graphs are symmetric: `a[v][w]` is always equal to `a[w][v]`. In C, it is easy to save space by storing only one-half of this symmetric matrix (see Exercise 17.20). At the extreme, we might consider using an array of bits (in this way, for instance, we could represent graphs of up to about 64,000 vertices in about 64 million 64-bit words) (see Exercise 17.21). These implementations have the slight complication that we need to add an ADT operation to test for the existence of an edge (see Exercise 17.19). (We do not use such an operation in our implementations because the code is slightly easier to understand when we test for the existence on an edge `v-w` by simply testing `a[v][w]`.) Such space-saving techniques are effective, but come at the cost of extra overhead that may fall in the inner loop in time-critical applications.

Many applications involve associating other information with each edge—in such cases, we can generalize the adjacency matrix to be an array that holds any information whatever. We reserve at least one value in the data type that we use for the array elements, to indicate that the indicated edge is absent. In Chapters 20 and 21, we explore the representation of such graphs.

Use of adjacency matrices depends on associating vertex names with integers between 0 and $V - 1$. This assignment might be done in one of many ways—for example, we consider a program that does so in Section 17.6). Therefore, the specific matrix of `0-1` values that

Program 17.5 Graph ADT output (adjacency-lists format)

Printing out the full adjacency matrix is unwieldy for sparse graphs, so we might choose to simply print out, for each vertex, the vertices that are connected to that vertex by an edge.

```
void GRAPHshow(Graph G)
  { int i, j;
    printf("%d vertices, %d edges\n", G->V, G->E);
    for (i = 0; i < G->V; i++)
      {
        printf("%2d:", i);
        for (j = 0; j < G->V; j++)
          if (G->adj[i][j] == 1) printf(" %2d", j);
        printf("\n");
      }
  }
```

we represent with a two-dimensional array in C is but one possible representation of any given graph as an adjacency matrix, because another program might give a different assignment of vertex names to the indices we use to specify rows and columns. Two arrays that appear to be markedly different could represent the same graph (see Exercise 17.17). This observation is a restatement of the graph isomorphism problem: Although we might like to determine whether or not two different arrays represent the same graph, no one has devised an algorithm that can always do so efficiently. This difficulty is fundamental. For example, our ability to find an efficient solution to various important graph-processing problems depends completely on the way in which the vertices are numbered (see, for example, Exercise 17.25).

Developing an ADT function that prints out the adjacency-matrix representation of a graph is a simple exercise (see Exercise 17.16). Program 17.5 illustrates a different implementation that may be preferred for sparse graphs: It just prints out the vertices adjacent to each vertex, as illustrated in Figure 17.9. These programs (and, specifically, their output) clearly illustrate a basic performance tradeoff. To print out the array, we need space for all V^2 entries; to print out the lists, we need room for just $V + E$ numbers. For sparse graphs, when V^2 is huge compared to $V + E$, we prefer the lists; for dense graphs, when E

```
 0:  1 2 5 6
 1:  0
 2:  0
 3:  4 5
 4:  3 5 6
 5:  0 3 4
 6:  0 4
 7:  8
 8:  7
 9:  10 11 12
10:  9
11:  9 12
12:  9 11
```

Figure 17.9 Adjacency lists format

This table illustrates yet another way to represent the graph in Figure 17.1: we associate each vertex with its set of adjacent vertices (those connected to it by a single edge). Each edge affects two sets: for every edge `u-v` *in the graph,* `u` *appears in* `v`*'s set and* `v` *appears in* `u`*'s set.*

and V^2 are comparable, we prefer the array. As we shall soon see, we make the same basic tradeoff when we compare the adjacency-matrix representation with its primary alternative: an explicit representation of the lists.

The adjacency-matrix representation is not satisfactory for huge sparse graphs: The array requires V^2 bits of storage and V^2 steps just to initialize. In a dense graph, when the number of edges (the number of `1` bits in the matrix) is proportional to V^2, this cost may be acceptable, because time proportional to V^2 is required to process the edges no matter what representation we use. In a sparse graph, however, just initializing the array could be the dominant factor in the running time of an algorithm. Moreover, we may not even have enough space for the matrix. For example, we may be faced with graphs with millions of vertices and tens of millions of edges, but we may not want—or be able—to pay the price of reserving space for *trillions* of `0` entries in the adjacency matrix.

On the other hand, when we do need to process a huge dense graph, then the `0`-entries that represent absent edges increase our space needs by only a constant factor and provide us with the ability to determine whether any particular edge is present with a single array access. For example, disallowing parallel edges is automatic in an adjacency matrix but is costly in some other representations. If we do have space available to hold an adjacency matrix, and either V^2 is so small as to represent a negligible amount of time or we will be running a complex algorithm that requires more than V^2 steps to complete, the adjacency-matrix representation may be the method of choice, no matter how dense the graph.

Exercises

▷ **17.16** Give an implementation of `GRAPHshow` for inclusion in the adjacency-lists graph ADT implementation (Program 17.3) that prints out a two-dimensional array of 0s and 1s like the one illustrated in Figure 17.7.

▷ **17.17** Give the adjacency-matrix representations of the three graphs depicted in Figure 17.2.

17.18 Given a graph, consider another graph that is identical to the first, except that the names of (integers corresponding to) two vertices are interchanged. How do the adjacency matrices of these two graphs differ?

▷ **17.19** Add a function `GRAPHedge` to the graph ADT that allows clients to test whether there is an edge connecting two given vertices, and provide an implementation for the adjacency-matrix representation.

▷ **17.20** Modify Program 17.3, augmented as described in Exercise 17.19, to cut its space requirements about in half by not including array entries `a[v][w]` for `w` greater than `v`.

17.21 Modify Program 17.3, augmented as described in Exercise 17.19, to use an array of bits, rather than of integers. That is, if your computer has B bits per word, your implementation should be able to represent a graph with V vertices in about V^2/B words (as opposed to V^2). Do empirical tests to assess the effect of using a bit array on the time required for the ADT operations.

17.22 Modify Program 17.4 to check `malloc` return codes and return `0` if there is insufficient memory available to represent the matrix.

17.23 Write a version of Program 17.4 that uses a single call to `malloc`.

17.24 Add implementations of `GRAPHcopy` and `GRAPHdestroy` to Program 17.3.

○ **17.25** Suppose that all k vertices in a group have consecutive indices. How can you determine from the adjacency matrix whether or not that group of vertices constitutes a clique? Add a function to the adjacency-matrix implementation of the graph ADT (Program 17.3) that finds, in time proportional to V^2, the largest group of vertices with consecutive indices that constitutes a clique.

17.4 Adjacency-Lists Representation

The standard representation that is preferred for graphs that are not dense is called the *adjacency-lists* representation, where we keep track of all the vertices connected to each vertex on a linked list that is associated with that vertex. We maintain an array of lists so that, given a vertex, we can immediately access its list; we use linked lists so that we can add new edges in constant time.

Program 17.6 is an implementation of the ADT interface in Program 17.1 that is based on this approach, and Figure 17.10 depicts an example. To add an edge connecting `v` and `w` to this representation of the graph, we add `w` to `v`'s adjacency list and `v` to `w`'s adjacency list. In this way, we still can add new edges in constant time, but the total amount of space that we use is proportional to the number of vertices plus the number of edges (as opposed to the number of vertices squared, for the adjacency-matrix representation). We again represent each edge in two different places: an edge connecting `v` and `w` is represented as nodes on both adjacency lists. It is important to include both; otherwise, we could not answer efficiently simple questions such as, "Which vertices are connected directly to vertex `v`?"

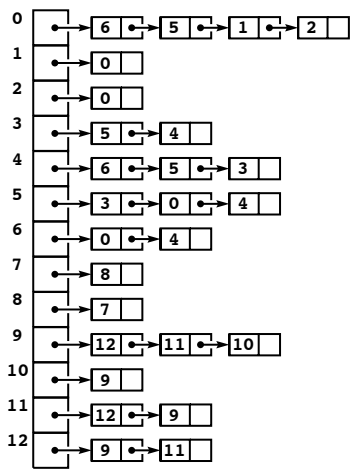


Figure 17.10
Adjacency-lists data structure

This figure depicts a representation of the graph in Figure 17.1 as an array of linked lists. The space used is proportional to the number of nodes plus the number of edges. To find the indices of the vertices connected to a given vertex v, we look at the vth position in an array, which contains a pointer to a linked list containing one node for each vertex connected to v. The order in which the nodes appear on the lists depends on the method that we use to construct the lists.

Program 17.6 Graph ADT implementation (adjacency lists)

This implementation of the interface in Program 17.1 uses an array of lists, one corresponding to each vertex. An edge `v-w` is represented by a node for `w` on list `v` and a node for `v` on list `w`. As in Program 17.3, `GRAPHedges` puts just one of the two representations of each edge into the output array. Implementations of `GRAPHcopy`, `GRAPHdestroy`, and `GRAPHremoveE` are omitted. The `GRAPHinsertE` code keeps insertion time constant by not checking for duplicate edges.

```
#include <stdlib.h>
#include "GRAPH.h"
typedef struct node *link;
struct node { int v; link next; };
struct graph { int V; int E; link *adj; };
link NEW(int v, link next)
  { link x = malloc(sizeof *x);
    x->v = v; x->next = next;
    return x;
  }
Graph GRAPHinit(int V)
  { int v;
    Graph G = malloc(sizeof *G);
    G->V = V; G->E = 0;
    G->adj = malloc(V*sizeof(link));
    for (v = 0; v < V; v++) G->adj[v] = NULL;
    return G;
  }
void GRAPHinsertE(Graph G, Edge e)
  { int v = e.v, w = e.w;
    G->adj[v] = NEW(w, G->adj[v]);
    G->adj[w] = NEW(v, G->adj[w]);
    G->E++;
  }
int GRAPHedges(Edge a[], Graph G)
  { int v, E = 0; link t;
    for (v = 0; v < G->V; v++)
      for (t = G->adj[v]; t != NULL; t = t->next)
        if (v < t->v) a[E++] = EDGE(v, t->v);
    return E;
  }
```

By contrast to Program 17.3, Program 17.6 builds multigraphs, because it does not remove parallel edges. Checking for duplicate edges in the adjacency-lists structure would necessitate searching through the lists and could take time proportional to V. Similarly, Program 17.6 does not include an implementation of the *remove edge* operation. Again, adding such an implementation is an easy exercise (see Exercise 17.28), but each deletion might take time proportional to V, to search through the two lists for the nodes to remove. These costs make the basic adjacency-lists representation unsuitable for applications involving huge graphs where parallel edges cannot be tolerated, or applications involving heavy use of the *remove edge* operation. In Section 17.5, we discuss the use of elementary data-structure techniques to augment adjacency lists such that they support constant-time *remove edge* and *parallel-edge detection* operations.

If a graph's vertex names are not integers, then (as with adjacency matrices) two different programs might associate vertex names with the integers from 0 to $V-1$ in two different ways, leading to two different adjacency-list structures (see, for example, Program 17.10). We cannot expect to be able to tell whether two different structures represent the same graph because of the difficulty of the graph isomorphism problem.

Moreover, with adjacency lists, there are numerous representations of a given graph even for a given vertex numbering. No matter in what order the edges appear on the adjacency lists, the adjacency-list structure represents the same graph (see Exercise 17.31). This characteristic of adjacency lists is important to know because the order in which edges appear on the adjacency lists affects, in turn, the order in which edges are processed by algorithms. That is, the adjacency-list structure determines how our various algorithms see the graph. Although an algorithm should produce a correct answer no matter how the edges are ordered on the adjacency lists, it might get to that answer by different sequences of computations for different orderings. If an algorithm does not need to examine all the graph's edges, this effect might affect the time that it takes. And, if there is more than one correct answer, different input orderings might lead to different output results.

The primary advantage of the adjacency-lists representation over the adjacency-matrix representation is that it always uses space pro-

portional to $E + V$, as opposed to V^2 in the adjacency matrix. The primary disadvantage is that testing for the existence of specific edges can take time proportional to V, as opposed to constant time in the adjacency matrix. These differences trace, essentially, to the difference between using linked lists and arrays to represent the set of vertices incident on each vertex.

Thus, we see again that an understanding of the basic properties of linked data structures and arrays is critical if we are to develop efficient graph ADT implementations. Our interest in these performance differences is that we want to avoid implementations that are inappropriately inefficient under unexpected circumstances when a wide range of operations is to be included in the ADT. In Section 17.5, we discuss the application of basic symbol-table algorithmic technology to realize many of the theoretical benefits of both structures. Nonetheless, because Program 17.6 is a simple implementation with the essential characteristics that we need to learn efficient algorithms for processing sparse graphs, we use it as the basis for many implementations in this book.

Exercises

▷ **17.26** Show, in the style of Figure 17.10, the adjacency-lists structure produced when you insert the edges in the graph

```
3-7 1-4 7-8 0-5 5-2 3-8 2-9 0-6 4-9 2-6 6-4
```

(in that order) into an initially empty graph, using Program 17.6.

17.27 Give implementations of `GRAPHshow` that have the same functionality as Exercise 17.16 and Program 17.5, for inclusion in the adjacency-lists graph ADT implementation (Program 17.6).

17.28 Provide an implementation of the *remove edge* function `GRAPHremoveE` for the adjacency-lists graph ADT implementation (Program 17.6). *Note*: Remember the possibility of duplicates.

17.29 Add implementations of `GRAPHcopy` and `GRAPHdestroy` to the adjacency-lists graph ADT implementation (Program 17.6).

○ **17.30** Give a simple example of an adjacency-lists graph representation that could not have been built by repeated addition of edges by Program 17.6.

17.31 How many different adjacency-lists representations represent the same graph as the one depicted in Figure 17.10?

17.32 Write a version of Program 17.6 that keeps the adjacency lists in sorted order of vertex index. Describe a situation where this approach would be useful.

○ **17.33** Add a function declaration to the graph ADT (Program 17.1) that removes self-loops and parallel edges. Provide the trivial implementation of this function for the adjacency-matrix–based ADT implementation (Program 17.3), and provide an implementation of the function for the adjacency-list–based ADT implementation (Program 17.6) that uses time proportional to E and extra space proportional to V.

17.34 Extend your solution to Exercise 17.33 to also remove degree-0 (isolated) vertices. *Note*: To remove vertices, you need to rename the other vertices and rebuild the data structures—you should do so just once.

• **17.35** Write an ADT function for the adjacency-lists representation (Program 17.6) that collapses paths that consist solely of degree-2 vertices. Specifically, every degree-2 vertex in a graph with no parallel edges appears on some path `u-...-w` where `u` and `w` are either equal or not of degree 2. Replace any such path with `u-w` and then remove all unused degree-2 vertices as in Exercise 17.34. *Note*: This operation may introduce self-loops and parallel edges, but it preserves the degrees of vertices that are not removed.

▷ **17.36** Give a (multi)graph that could result from applying the transformation described in Exercise 17.35 on the sample graph in Figure 17.1.

17.5 Variations, Extensions, and Costs

In this section, we describe a number of options for improving the graph representations discussed in Sections 17.3 and 17.5. The modifications fall into one of two categories. First, the basic adjacency-matrix and adjacency-lists mechanisms extend readily to allow us to represent other types of graphs. In the relevant chapters, we consider these extensions in detail, and give examples; here, we look at them briefly. Second, we often need to modify or augment the basic data structures to make certain operations more efficient. We do so as a matter of course in the chapters that follow; in this section, we discuss the application of data-structure design techniques to enable efficient implementation of several basic functions.

For *digraphs*, we represent each edge just once, as illustrated in Figure 17.11. An edge `v-w` in a digraph is represented by a 1 in the entry in row `v` and column `w` in the adjacency array or by the appearance of `w` on `v`'s adjacency list in the adjacency-lists representation. These representations are simpler than the corresponding representations that we have been considering for undirected graphs, but the asymmetry makes digraphs more complicated combinatorial objects than undirected graphs, as we see in Chapter 19. For example, the

	0	1	2	3	4	5	6	7	8	9	10	11	12
0	1	1	1	0	0	1	1	0	0	0	0	0	0
1	0	1	0	0	0	0	0	0	0	0	0	0	0
2	0	0	1	0	0	0	0	0	0	0	0	0	0
3	0	0	0	1	0	0	0	0	0	0	0	0	0
4	0	0	0	1	1	0	0	0	0	0	0	0	0
5	0	0	0	1	1	1	0	0	0	0	0	0	0
6	0	0	0	0	1	0	1	0	0	0	0	0	0
7	0	0	0	0	0	0	0	1	1	0	0	0	0
8	0	0	0	0	0	0	0	0	1	0	0	0	0
9	0	0	0	0	0	0	0	0	0	1	1	1	1
10	0	0	0	0	0	0	0	0	0	0	1	0	0
11	0	0	0	0	0	0	0	0	0	0	0	1	1
12	0	0	0	0	0	0	0	0	0	0	0	0	1

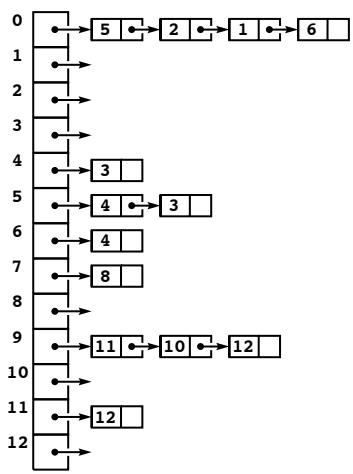


Figure 17.11
Digraph representations

The adjacency-array and adjacency-lists representations of a digraph have only one representation of each edge, as illustrated in the adjacency array (top) *and adjacency lists* (bottom) *representation of the set of edges in Figure 17.1 interpreted as a digraph (see Figure 17.6, top).*

standard adjacency-lists representation gives no direct way to find all edges coming into a vertex in a digraph, so we must make appropriate modifications if that operation needs to be supported.

For *weighted graphs* and *networks*, we fill the adjacency matrix with weights instead of Boolean values (using some nonexistent weight to represent the absence of an edge); in the adjacency-lists representation, we include a vertex weight in the adjacency structure or an edge weight in adjacency-list elements.

It is often necessary to associate still more information with the vertices or edges of a graph, to allow that graph to model more complicated objects. We can associate extra information with each edge by extending the `Edge` type in Program 17.1 as appropriate, then using instances of that type in the adjacency matrix, or in the list nodes in the adjacency lists. Or, since vertex names are integers between 0 and $V-1$, we can use vertex-indexed arrays to associate extra information for vertices, perhaps using an appropriate ADT. Alternatively, we can simply use a separate symbol table ADT to associate extra information with each vertex and edge (see Exercise 17.46 and Program 17.10).

To handle various specialized graph-processing problems, we often need to add specialized auxiliary data structures to the graph ADT. The most common such data structure is a vertex-indexed array, as we saw already in Chapter 1, where we used vertex-indexed arrays to answer connectivity queries. We use vertex-indexed arrays in numerous implementations throughout the book.

As an example, suppose that we wish to know whether a vertex v in a graph is isolated. Is v of degree 0? For the adjacency-lists representation, we can find this information immediately, simply by checking whether `adj[v]` is null. But for the adjacency-matrix representation, we need to check all V entries in the row or column corresponding to `v` to know that each one is not connected to any other vertex; and for the array-of-edges representation, we have no better approach than to check all E edges to see whether there are any that involve `v`. Instead of these potentially time-consuming computations, we could implement a simple online algorithm that maintains a vertex-indexed array such that we can find the degree of any vertex in constant time (see Exercise 17.40). Such a substantial performance differential for such a simple problem is typical in graph processing.